



Tipos de *datasets* en Kedro (CSV, JSON, Parquet, SQL, etc.)

Kedro es compatible con una amplia variedad de formatos y fuentes de datos mediante la biblioteca **kedro-datasets**, la cual ofrece implementaciones para distintos formatos de almacenamiento ampliamente utilizados.

Cada tipo de *dataset* está diseñado para gestionar operaciones de lectura y escritura de manera transparente dentro del *pipeline*.

A continuación observarás los tipos de *dataset* más utilizados.

Tipos de *dataset* para archivos tabulares o jerárquicos

Tipo de <i>Dataset</i>	Descripción	Ejemplo de uso
pandas.CSVDataset	Carga y escribe archivos CSV usando pandas.	Datos tabulares simples en formato de texto plano separado por comas.
pandas.ExcelDataset	Carga y escribe archivos Excel usando pandas.	Datos en formato tabular propietario, de uso extensivo en la industria.
pandas.ParquetDataset	Carga y escribe archivos Parquet usando pandas.	Almacenamiento eficiente de tablas en archivos binarios, guardan el tipo de dato de cada columna.
pandas.HDFDataset	Carga y escribe archivos HDF5 usando pandas.	Datos científicos jerárquicos.
pandas.JSONDataset	Carga y escribe JSON usando pandas.	Datos semiestructurados en formato de texto plano, de uso común en interfaces y aplicaciones web.

Tipos de *dataset* para bases de datos SQL

Tipo de <i>Dataset</i>	Descripción
<code>pandas.SQLTableDataset</code>	Carga y escribe datos en tablas de bases de datos SQL usando pandas.
<code>pandas.SQLQueryDataset</code>	Ejecuta consultas SQL y carga los resultados usando pandas.

Tipos de *dataset* para datos de *machine learning*

Tipo de <i>Dataset</i>	Descripción
<code>pickle.PickleDataset</code>	Utiliza el formato nativo Pickle para guardar cualquier objeto de Python.
<code>tensorflow. TensorFlowModelDataset</code>	Modelos de TensorFlow.
<code>sklearn.SKLearnDataset</code>	Modelos de scikit-learn.

Tipo de <i>Dataset</i>	Descripción
MemoryDataset	Almacenamiento en memoria (no persistente).
api.APIDataset	Conexión a APIs externas.
text.TextDataset	Archivos de texto simples.
matplotlib.MatplotlibWriter	Para guardar figuras de matplotlib.

Cada tipo de *dataset* tiene parámetros y especificaciones únicas. La lista oficial de tipos de *datasets* actualizada se puede encontrar en <https://docs.kedro.org/projects/kedro-datasets/en/latest/>, donde se documenta cada tipo de *dataset* con sus parámetros, configuraciones y ejemplos de uso.



Ejemplo de configuración para diferentes tipos de *datasets*

A continuación, se muestra un ejemplo de cómo configurar diferentes tipos de *datasets* en el `catalog.yml`:

```
# Dataset CSV
ventas:
  type: pandas.CSVDataset
  filepath: data/01_raw/ventas.csv
  load_args:
    sep: ','
    na_values: ['N/A', 'NA']

# Dataset JSON
usuarios:
  type: pandas.JSONDataset
  filepath: data/01_raw/usuarios.json

# Dataset Parquet
ventas_procesadas:
  type: pandas.ParquetDataset
  filepath: data/02_intermediate/ventas_procesadas.parquet

# Dataset SQL
clientes:
  type: pandas.SQLTableDataset
  table_name: clientes
  credentials: db_credentials

# Dataset para modelo ML
modelo_prediccion:
  type: pickle.PickleDataset
  filepath: data/06_models/modelo_prediccion.pkl
  versioned: true
```

Figura 1. Ejemplo de configuración de *datasets*.

Cada tipo de *dataset* carga los datos en un formato específico (por ejemplo, `Pandas.DataFrame`). Mientras se considere este formato en el desarrollo de los nodos, se puede intercambiar el origen físico del dato sin cambios en el código del nodo.

También es importante notar que para tipos de *datasets* como `SQLTableDataset` que pueden requerir credenciales, estas se pueden especificar en el archivo `conf/local/credentials.yml`, que no se incluye en el control de versiones por seguridad.

Por último, cuando un *dataset* utilizado como destino se configura con versionado (**versioned: true**), se guardará una versión cada vez que el *pipeline* se ejecute. Esta versión se identifica con el nombre del *dataset* en el catálogo y el sello de tiempo del archivo. Cabe recalcar que no todos los tipos de *dataset* soportan versionado.

En https://docs.kedro.org/en/stable/data/data_catalog_yaml_examples.html puedes encontrar ejemplos de cómo configurar *datasets* para varios tipos de fuentes y destinos de datos.

Configuración de rutas de datos en catalog.yml

La configuración de rutas de datos en el archivo catalog.yml es flexible y puede adaptarse a diversas necesidades.

Kedro emplea el paquete **fsspec** para acceder a distintos sistemas de almacenamiento, lo que permite trabajar con sistemas de archivos locales, archivos en red o soluciones de almacenamiento en la nube, entre otros.

Las rutas dentro de **catalog.yml** pueden definirse utilizando distintos protocolos. Por ejemplo:

1. Sistema de archivos local:

```
dataset:
  type: pandas.CSVDataset
  filepath: data/01_raw/data.csv
```

2. Amazon S3:

```
dataset:  
type: pandas.CSVDataset  
filepath: s3://mi-bucket/data/01_raw/data.csv  
credentials: s3_credentials
```

3. Google Cloud Storage:

```
dataset:  
type: pandas.CSVDataset  
filepath: gcs://mi-bucket/data/01_raw/data.csv  
credentials: gcs_credentials
```

4. Azure Blob Storage:

```
dataset:  
type: pandas.CSVDataset  
filepath: abfs://container/data/01_raw/data.csv  
credentials: azure_credentials
```

5. HDFS (Hadoop Distributed File System):

```
dataset:  
type: pandas.CSVDataset  
filepath: hdfs://namenode:port/path/to/data.csv
```

Buenas prácticas para estructurar rutas de datos

En proyectos Kedro, se recomienda seguir una estructura de carpetas estándar para organizar los datos:

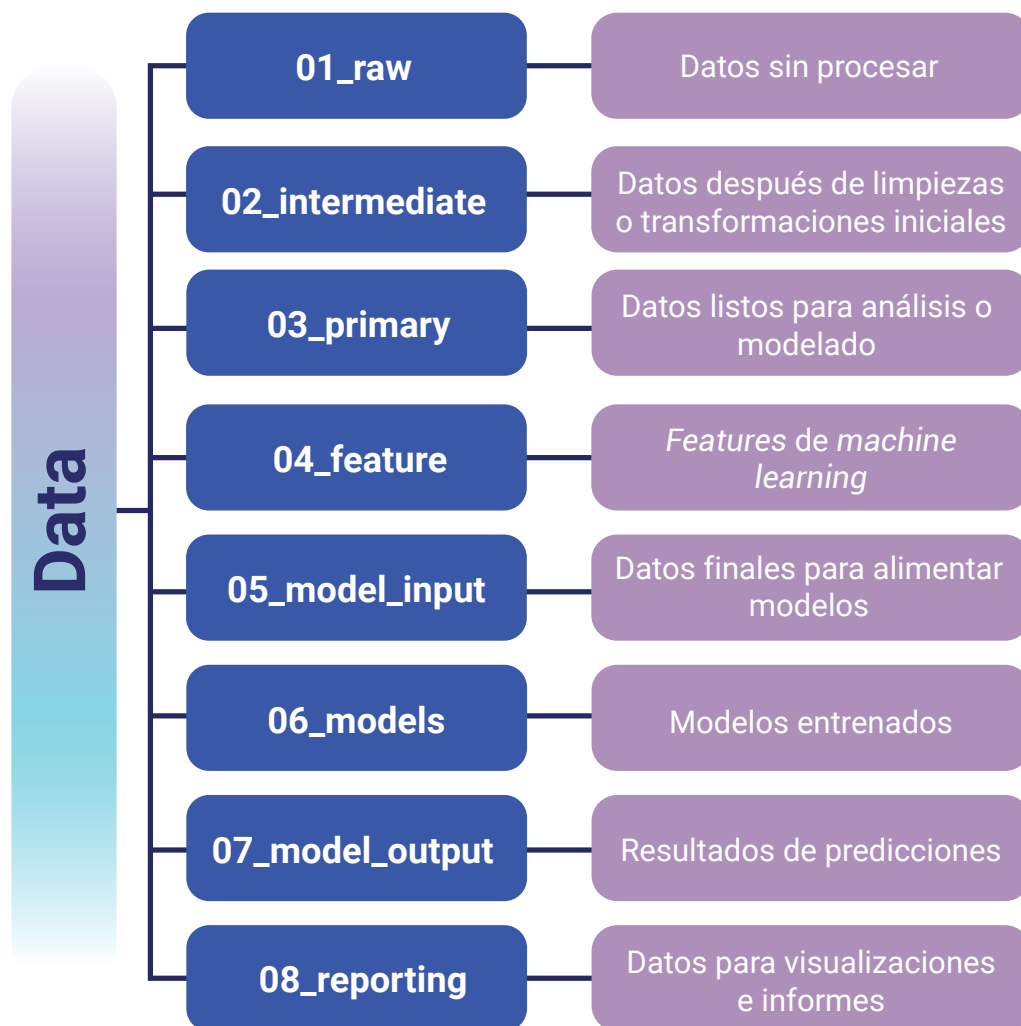


Figura 2. Buenas prácticas para estructurar rutas de datos.

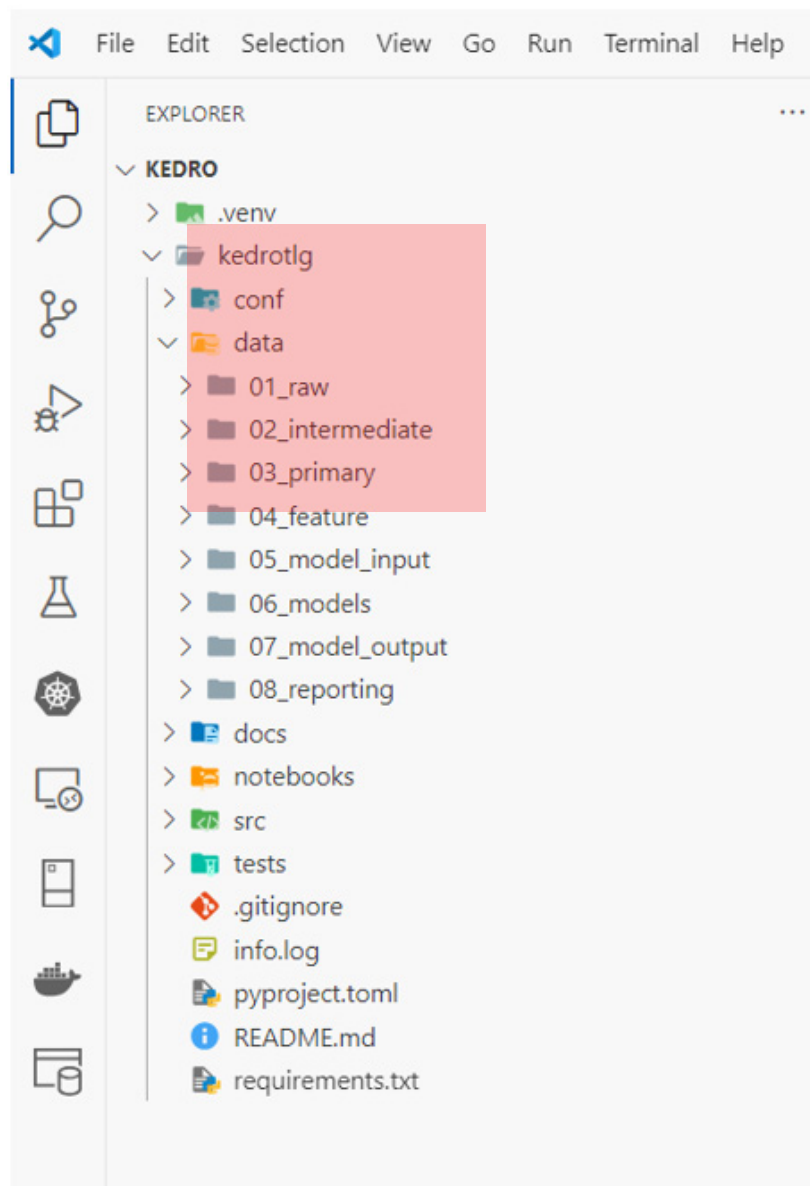


Figura 3. Estructura de carpetas estándar.

Siguiendo esta estructura, se podría configurar un catálogo de manera organizada, por ejemplo:

```
# Datos de entrada sin procesar
raw_data:
  type: pandas.CSVDataset
  filepath: data/01_raw/data.csv

# Datos después del preprocesamiento
preprocessed_data:
  type: pandas.ParquetDataset
  filepath: data/02_intermediate/preprocessed_data.parquet
# Características para el modelo
features:
  type: pandas.ParquetDataset
  filepath: data/04_feature/features.parquet

# Modelo entrenado
model:
  type: pickle.PickleDataset
  filepath: data/06_models/model.pkl
  versioned: true

# Resultados de predicción
predictions:
  type: pandas.CSVDataset
  filepath: data/07_model_output/predictions.csv
  save_args:
    index: false
```

Es recomendable usar rutas relativas en lugar de absolutas para garantizar la portabilidad del proyecto entre diferentes computadoras o sistemas operativos. Kedro interpreta las rutas relativas tomando como referencia el directorio raíz del proyecto.

Separación de entornos (conf/local, conf/dev, conf/prod)

Otra ventaja que tiene Kedro, es que permite definir diferentes configuraciones para distintos entornos, lo que facilita la transición entre desarrollo, pruebas y producción.

Por defecto, un proyecto Kedro incluye dos entornos principales:

- Base: configuración base que se comparte entre todos los entornos.
- Local: configuración local que sobrescribe la configuración base (no se incluye en control de versiones).

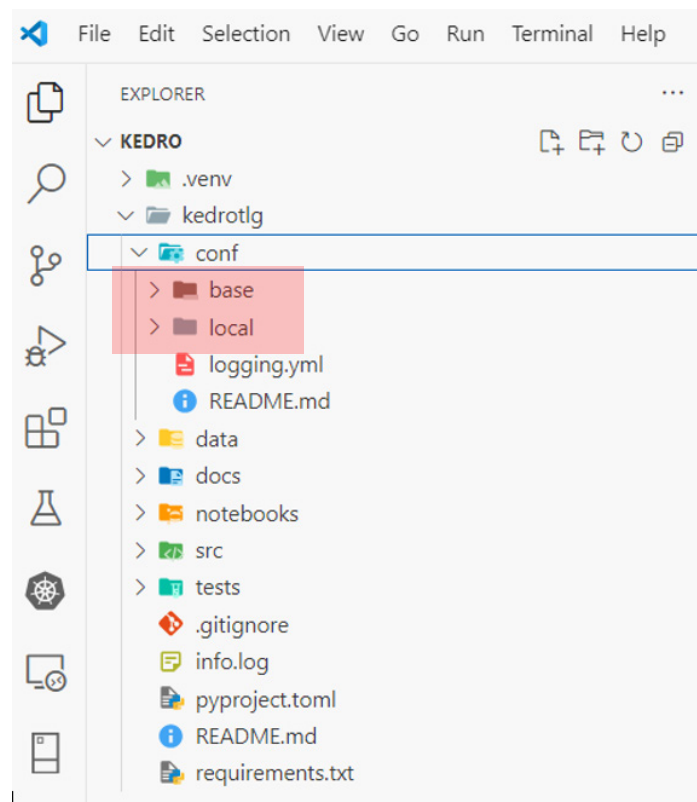


Figura 4. Carpetas de configuraciones.

Cada ambiente se configura en su propia subcarpeta dentro de la carpeta **conf**.

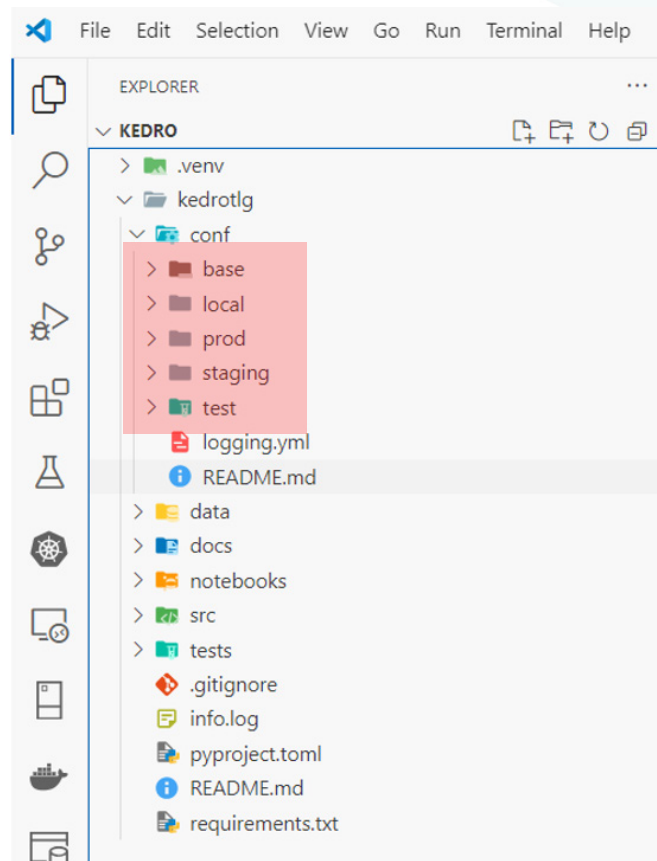


Figura 5. Subcarpetas.

Kedro carga la configuración en un orden específico:

1. Primero carga los archivos del entorno `base`.
2. Luego carga los archivos del entorno especificado (por defecto, `local`).
3. Los archivos del entorno especificado sobrescriben cualquier configuración del entorno `base` si hay conflictos.

De esta manera se pueden especificar distintos archivos para el mismo *dataset*, dependiendo del tipo de ambiente.

Para ejemplificar, podríamos tener lo siguiente en el proyecto:

Configuración en **conf/base/catalog.yml**:

```
genres:
  type: pandas.CSVDataset
  filepath: data/01_raw/genres_v2.csv

playlists:
  type: pandas.CSVDataset
  filepath: data/01_raw/playlists.csv
```

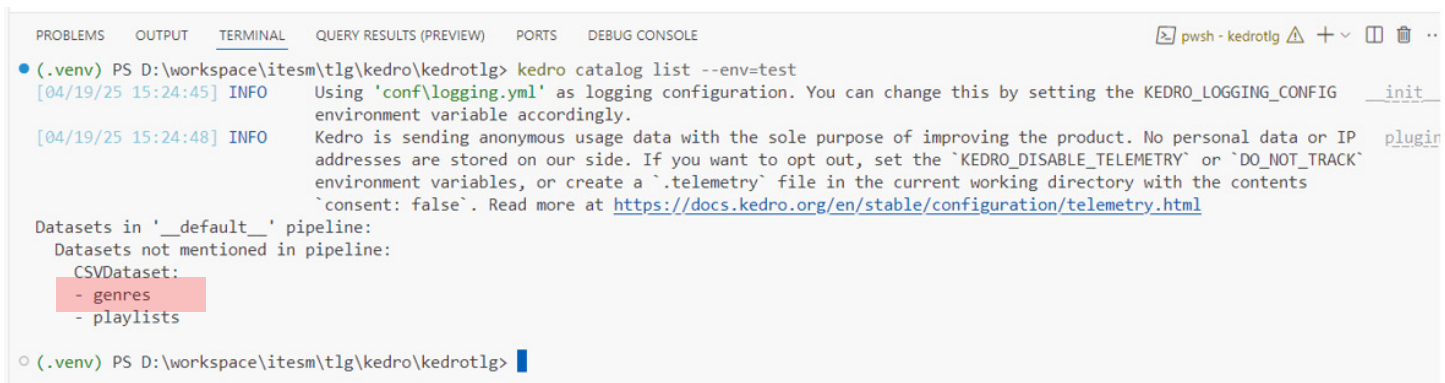
Configuración en **conf/test/catalog.yml**:

```
genres:
  type: pandas.CSVDataset
  filepath: data/01_raw/genres_v2_sample.csv
```

Asumiendo que existe el archivo **data/01_raw/genres_v2_sample.csv** (puede ser una muestra del archivo base).

Para ejecutar Kedro usando un entorno específico, usa la opción **--env**:

```
kedro catalog list --env=test
```



```
PROBLEMS OUTPUT TERMINAL QUERY RESULTS (PREVIEW) PORTS DEBUG CONSOLE
(.venv) PS D:\workspace\itesm\tlg\kedro\kedrotlg> kedro catalog list --env=test
[04/19/25 15:24:45] INFO Using 'conf\logging.yml' as logging configuration. You can change this by setting the KEDRO_LOGGING_CONFIG environment variable accordingly.
[04/19/25 15:24:48] INFO Kedro is sending anonymous usage data with the sole purpose of improving the product. No personal data or IP addresses are stored on our side. If you want to opt out, set the 'KEDRO_DISABLE_TELEMETRY' or 'DO_NOT_TRACK' environment variables, or create a '.telemetry' file in the current working directory with the contents 'consent: false'. Read more at https://docs.kedro.org/en/stable/configuration/telemetry.html

Datasets in '__default__' pipeline:
Datasets not mentioned in pipeline:
  CSVDataset:
    - genres
    - playlists

(.venv) PS D:\workspace\itesm\tlg\kedro\kedrotlg>
```

Figura 6. Kedro en entorno específico.

En este caso, Kedro cargará el *dataset playlists* desde el `catalog.yml` en la configuración `base`, pero *genres* lo cargará desde el `catalog.yml` en la configuración en la carpeta `test`. Lo mismo aplicará si existen archivos `credentials.yml`. Es importante recalcar, que los archivos `credentials.yml` se excluyen automáticamente de subirse al repositorio de Git.

Si no se especifica el entorno, Kedro usará `local` por defecto.

Uso de `parameters.yml` en Kedro

Los parámetros en Kedro son datos o configuraciones que pueden ser utilizadas por los nodos de un *pipeline*, pero no se leen desde *datasets*. Se utilizan como hiperparámetros de modelos, umbrales, o cualquier otro valor configurable que quieras mantener fuera del código.

Los parámetros se definen en archivos de configuración con el prefijo *parameters*, normalmente en `conf/base/parameters.yml`. Su estructura es simplemente un documento YAML:

```
# conf/base/parameters.yml
train_test_split:
  test_size: 0.2
  random_state: 42
model_options:
  n_estimators: 100
  max_depth: 10
  learning_rate: 0.01
preprocessing:
  outlier_threshold: 3.0
  fill_strategy: "median"
```

Kedro intentará convertir los valores proporcionados al tipo de dato apropiado:

- Si un valor se puede convertir a número (entero o flotante), Kedro lo hará automáticamente.
- De lo contrario, se tratará como una cadena de texto.

Al igual que con el catálogo y las credenciales, puedes tener diferentes archivos de parámetros para diferentes entornos:

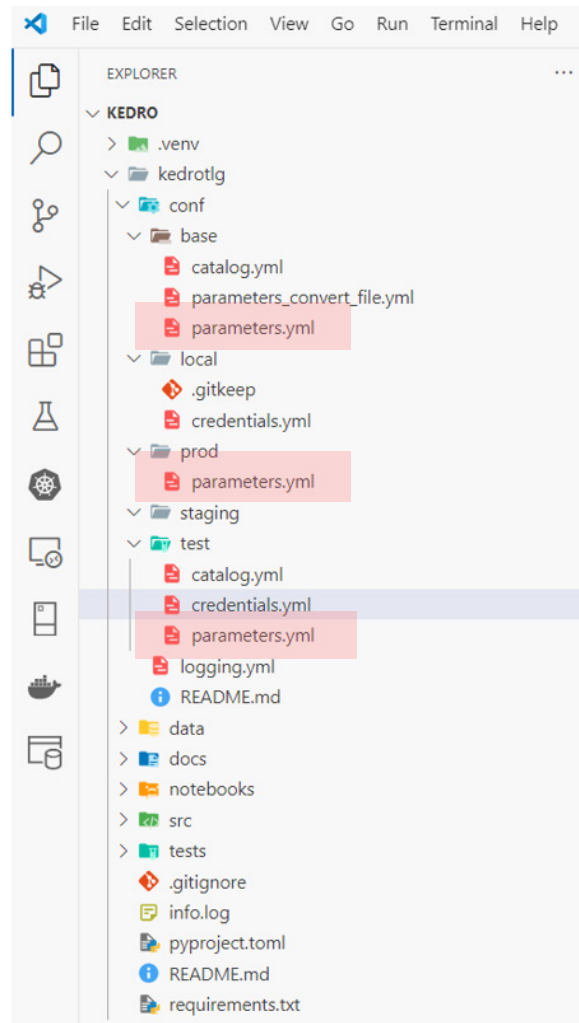


Figura 7. Archivos de parámetros para diferentes entornos.

Es recomendable organizar los parámetros en grupos lógicos para facilitar su mantenimiento. Por ejemplo:

```
# Parámetros para la fase de datos
data:
  train_fraction: 0.8
  random_seed: 42
  features:
    - feature_1
    - feature_2
    - feature_3
# Parámetros para la fase de modelado
model:
  algorithm: "random_forest"
  hyperparameters:
    n_estimators: 100
    max_depth : 10
    min_samples_split: 5
    random_state: 42
# Parámetros para la evaluación
evaluation:
  metrics:
    - accuracy
    - precision
    - recall
  threshold: 0.5
```

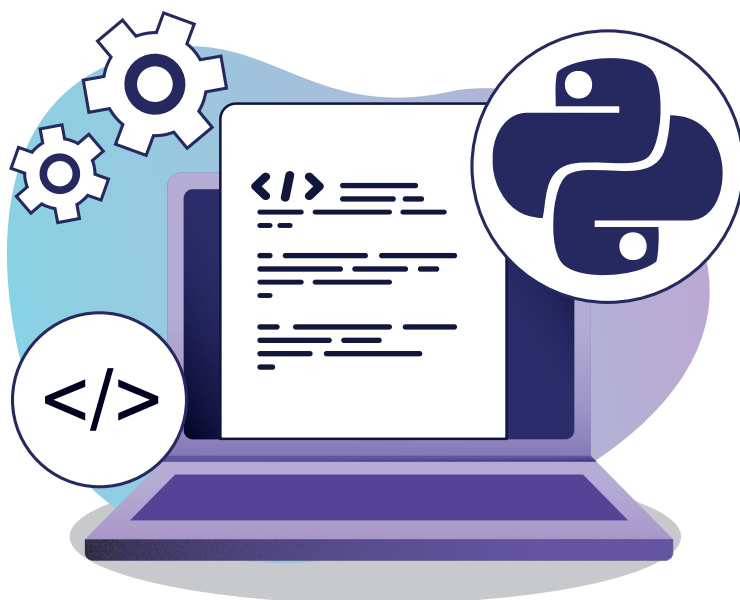
Esta estructura jerárquica permite que sea más fácil acceder a grupos específicos de parámetros en tus nodos.

Puedes crear archivos de parámetros específicos para diferentes *pipelines* en tu proyecto. Por ejemplo:

- `parameters.yml`: parámetros generales.
- `parameters_data_processing.yml`: parámetros específicos para el *pipeline* de nombre `data_processing`.
- `parameters_data_science.yml`: parámetros específicos para el *pipeline* `data_science`.

Estos parámetros se combinarán en tiempo de ejecución, así que hay que poner especial cuidado en no tener claves duplicadas entre archivos del mismo entorno, ya que esto generará un error.

Los parámetros se pueden acceder desde los nodos (en los siguientes temas veremos que es un nodo) como un diccionario de Python.



Sobrescritura de parámetros desde la línea de comandos

Kedro también permite sobrescribir parámetros al ejecutar un *pipeline*, sin necesidad de modificar los archivos de configuración. Esto es particularmente útil para experimentos rápidos o para ejecutar *pipelines* con diferentes configuraciones. Para establecer un parámetro, utiliza la opción `--params=` desde la línea de comandos, con cada par llave=valor separada por coma (sin espacios), por ejemplo:

```
kedro run --params=param_key1=value1,param_key2=value2
```

Para parámetros anidados, puedes usar notación de punto para acceder a los niveles profundos:

```
kedro run --params=model.hyperparameters.max_depth=15
```

Si los valores contienen espacios o caracteres especiales, se deberá encerrar toda la cadena de parámetros entre comillas:

```
kedro run --params="preprocessing.text_col='título del artículo',model_options.algo_name=RandomForest"
```

Cabe destacar que establecer parámetros desde la línea de comandos tiene algunas limitaciones y consideraciones:

1. **Precedencia:** los parámetros de línea de comandos tienen precedencia sobre los definidos en archivos de configuración. Es decir, los parámetros de línea de comandos reemplazan completamente los parámetros correspondientes en los archivos de configuración.
2. **No para producción:** esta característica es más adecuada para experimentación y desarrollo, no para entornos de producción, donde es preferible tener configuraciones explícitas y versionadas.
3. **Documentación:** es una buena práctica documentar los experimentos realizados con diferentes parámetros para referencia futura.

Hasta el momento, hemos visto cómo se definen las fuentes y destinos de datos, incluyendo parámetros de operación, en Kedro.

Entender la configuración del *Data Catalog* es fundamental para poder diseñar, implementar y operar un *pipeline* en Kedro.

- ¿Puedes identificar las fuentes y destinos de datos en tus proyectos?
- ¿Cuál será el tipo de *dataset* de Kedro más adecuado para cada fuente y destino?

Material adicional

Kedro. (s.f.). *Conozca Kedro*. Kedro. Recuperado el 01 de abril de 2025 de: <https://docs.kedro.org/en/stable/index.html>

Kedro. (s.f.). *Parameters*. [Parámetros]. Kedro. Recuperado el 01 de abril de 2025 de: <https://docs.kedro.org/en/stable/configuration/parameters.html#how-to-specify-parameters-at-runtime>

Python. (s.f.). *venv — Creación de entornos virtuales*. Python. Recuperado el 01 de abril de 2025 de: <https://docs.python.org/es/3.13/library/venv.html>

Visual Studio Code. (s.f.). *Tutorial: Introducción a Visual Studio Code*. Visual Studio Code. Recuperado el 01 de abril de 2025 de: <https://code.visualstudio.com/docs/getstarted/getting-started>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

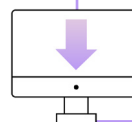
Bibliografía

Kedro. (s.f.). *Create a new Kedro project*. [Crea un nuevo Proyecto en Kedro]. Kedro. Recuperado el 01 de abril de 2025 de: https://docs.kedro.org/en/1.1.1/create/new_project/

Kedro. (s.f.). *Introduction to the Data Catalog*. [Introducción al Data Catalog]. Kedro. Recuperado el 01 de abril de 2025 de: https://docs.kedro.org/en/1.1.1/catalog-data/data_catalog/

PyPI. (s.f.). *Kedro*. Kedro. Recuperado el 01 de abril de 2025 de: <https://pypi.org/project/kedro/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “Archivos adjuntos” de la plataforma.